```
from anthropic import Anthropic
from dotenv import load_dotenv
import os
import subprocess
from pathlib import Path

load_dotenv(override=True)
client=Anthropic(base_url=os.getenv("ANTHROPIC_BASE_URL"))
MODEL=os.environ["MODEL_ID"]

WORKDIR=Path(os.getenv("AGENT_WORKDIR",os.getcwd())).resolve()
WORKDIR.mkdir(parents=True,exist_ok=True)

SYSTEM=f"you are a coding agent at {WORKDIR}.Use tools to solve tasks.act.don't explain"


TOOLS=[
    {
        "name":"bash",
        "description":"Run a shell command",
        "input_schema":{
            "type":"object",
            "properties":{"command":{"type":"string","description":"Shell command to
execute"}},
            "required":["command"]
        }
    },
    {
        "name":"read_file",
        "description":"Read file contents",
        "input_schema":{
            "type":"object",
            "properties":{"path":{"type":"string","description":"Relative path to file"},
                          "limit":{"type":"integer","description":"Max lines to
read,optional"}},
            "required":["path"]
        }
    },
    {
        "name":"write_file",
        "description":"Write content to file",
        "input_schema":{
            "type":"object",
            "properties":{"path":{"type":"string","description":"Relative path to file"},
                          "content":{"type":"string","description":"File content to
write"}},
            "required":["path",'content']
        }
    },
    {
        "name":"edit_file",
        "description":"Replace exact text in a file once",
```

```
        "input_schema":{
            "type":"object",
            "properties":{"path":{"type":"string","description":"Relative path to file"},
                          "old_text":{"type":"string","description":"Text to replace"},
                          "new_text":{"type":"string","description":"Replacement text"}},
            "required":["path",'old_text',"new_text"]
        }
    },
    {
        "name":"glob",
        "description":"Find files matching a glob pattern; **matches recursively",
        "input_schema":{
            "type":"object",
            "properties":{"pattern":{"type":"string","description":"Glob pattern, e.g.
**/*.py"}
                          },
            "required":["pattern"]
        }
    },

]

# 工具内部管理
def safe_path(p:str)->Path:
    path=(WORKDIR/p).resolve()
    if not path.is_relative_to(WORKDIR):
        raise ValueError(f"Path escapes workspace:{p}")
    return path

# 工具定义
def run_bash(command:str)->str:
    dangerous=["rm -rf /","sudo","shutdown","reboot",">/dev/"]
    if any(d in command for d in dangerous):
        return "Error:Dangerous command blocked"
    try:
        r=subprocess.run(command,shell=True,cwd=WORKDIR,
                         capture_output=True,text=True,errors="replace",
                         timeout=120)
        out=(r.stderr+r.stdout).strip()
        return out[:50000] if out else "no output"
    except subprocess.TimeoutExpired:
        return "Error:Timeout(120s)"
    except (FileNotFoundError,OSError) as e:
        return f"Error:{e}"

def run_read(path:str,limit:int|None=None)->str:
    try:
        lines=safe_path(path).read_text(encoding="utf-8").splitlines()
        if limit and limit<len(lines):
            lines=lines[:limit]+[f"...({len(lines)-limit}) more lines"]
        return "\n".join(lines)
    except Exception as e:
        return f"Error:{e}"
```

```python
def run_write(path:str,content:str)->str:
    try:
        file_path=safe_path(path)
        file_path.parent.mkdir(parents=True,exist_ok=True)
        file_path.write_text(content,encoding="utf-8")
        return f"wrote {len(content)} bytes to {path}"
    except Exception as e:
        return f"Error :{e}"

def run_edit(path:str,old_text:str,new_text:str)->str:
    try:
        file_path=safe_path(path)
        text=file_path.read_text(encoding="utf-8")
        if old_text not in text:
            return f"Error:text not found in {path}"
        file_path.write_text(text.replace(old_text,new_text,1),encoding="utf-8")
        return f"Edited {path}"
    except Exception as e:
        return f"Error:{e}"

def run_glob(pattern:str)->str:
    import glob as g
    try:
        matches=sorted({
            match for match in g.glob(pattern,root_dir=WORKDIR,recursive=True)
            if (WORKDIR/match).resolve().is_relative_to(WORKDIR)
        })
        shown=matches[:200]
        if len(matches)>200:
            shown.append("...(more matches omitted;narrow the pattern)")
            return "\n".join(shown) if shown else "(no matches)"
    except Exception as e:
        return f"Error:{e}"

# 工具注册表
TOOL_HANDLERS={
    "bash":run_bash,
    "read_file":run_read,
    "write_file":run_write,
    "edit_file":run_edit,
    "glob":run_glob
}


# 权限管理
# 防御bash 硬拒绝表
DENY_LIST = ["rm -rf /", "sudo", "shutdown", "reboot", "mkfs", "dd if=", "> /dev/sda"]

def check_deny_list(command: str) -> str | None:
    for pattern in DENY_LIST:
        if pattern in command:
            return f"Blocked: '{pattern}' is on the deny list"
```

```
    return None

# 规则匹配 防御其他工具，看路径
PERMISSION_RULES = [
    {"tools": ["read_file", "write_file", "edit_file"],
     "check": lambda args: not (WORKDIR / args.get("path",
"")).resolve().is_relative_to(WORKDIR),
     "message": "Access outside workspace"},
    {"tools": ["bash"],
     "check": lambda args: any(kw in args.get("command", "") for kw in ["rm ", "> /etc/",
"chmod 777"]),
     "message": "Potentially destructive command"},
]

def check_rules(tool_name: str, args: dict) -> str | None:
    for rule in PERMISSION_RULES:
        if tool_name in rule["tools"] and rule["check"](args):
            return rule["message"]
    return None


# 人工审核
def ask_user(tool_name: str, args: dict, reason: str) -> str:
    print(f"\n\033[33m[permission] {reason}\033[0m")
    print(f"   Tool: {tool_name}({args})")
    choice = input("   Allow? [y/N] ").strip().lower()
    return "allow" if choice in ("y", "yes") else "deny"


# 整体权限审核代码
def check_permission(block) -> bool:
    # 闸门 1: 硬拒绝(只管 bash,黑名单里全是 shell 命令)
    if block.name == "bash":
        reason = check_deny_list(block.input.get("command", ""))
        if reason:
            print(f"\n\033[31m[blocked] {reason}\033[0m")
            return False

    # 闸门 2 + 3: 规则匹配 → 命中就问人
    reason = check_rules(block.name, block.input)
    if reason:
        decision = ask_user(block.name, block.input, reason)
        if decision == "deny":
            return False

    return True

# 挂钩系统
HOOKS = {
    "UserPromptSubmit": [],   # 用户输入提交后、进 LLM 之前
    "PreToolUse":       [],   # 工具执行之前
    "PostToolUse":      [],   # 工具执行之后
    "Stop":             [],   # 循环即将退出时
```

```
}

def register_hook(event: str, callback):
    HOOKS[event].append(callback)

def trigger_hooks(event: str, *args):
    for callback in HOOKS[event]:
        result = callback(*args)
        if result is not None:      # 返回非 None → 有话说，立即中断
            return result
    return None


# 日志hook（PreToolUse）——审计需求落地，两行：返回 None：日志只看不管，永远不拦。
def log_hook(block):
    args_preview = str(list(block.input.values())[:2])[:60]
    print(f"\033[90m[HOOK] {block.name}({args_preview})\033[0m")
    return None

register_hook("PreToolUse", log_hook)

def permission_hook(block):
    if not check_permission(block):         # 上一篇的三道闸门，原样在里面
        return "Permission denied."         # 非 None → 拦下，拒绝回流给模型
    return None                             # None → 放行

register_hook("PreToolUse", permission_hook)


# 自动暂存 hook（PostToolUse）——留痕需求落地：
def auto_git_add_hook(block, output):
    if block.name in ("write_file", "edit_file"):
        path = block.input.get("path", "")
        subprocess.run(f"git add {path}", shell=True, cwd=WORKDIR)
    return None

register_hook("PostToolUse", auto_git_add_hook)

# 大输出警报 hook（PostToolUse）——工具输出超十万字符就亮黄灯，提醒你该给 bash 加截断了：
def large_output_hook(block, output):
    if len(str(output)) > 100000:
        print(f"\033[33m[HOOK] ⚠ {block.name} 输出 {len(str(output))} 字符，注意截断\033[0m")
    return None

#会话统计 hook（Stop）——收尾需求落地：
def summary_hook(messages):
    tool_count = sum(
        1 for m in messages
        if isinstance(m.get("content"), list)
        for b in m["content"]
        if isinstance(b, dict) and b.get("type") == "tool_result"
    )
    print(f"\033[90m[HOOK] 本会话共 {tool_count} 次工具调用\033[0m")
```

```
    return None

register_hook("Stop", summary_hook)


# 加一个 UserPromptSubmit 的，每次输入前打一行当前工作目录，让你随时知道它在哪个目录里干活：
def context_hook(query):
    print(f"\033[90m[HOOK] cwd = {WORKDIR}\033[0m")
    return None

register_hook("UserPromptSubmit", context_hook)


STATE = {"tests_ran": False}

def test_tracker_hook(block, output):
    """PostToolUse: 留意本会话跑没跑过测试"""
    if block.name == "bash" and "pytest" in block.input.get("command", ""):
        STATE["tests_ran"] = True
    return None

def quality_gate_hook(messages):
    """Stop: 改了代码没跑测试？那不能下班"""
    if not STATE["tests_ran"]:
        return ("你改了代码但还没有运行 pytest 验证。"
                "请先运行测试，全部通过后再结束任务。")
    return None

register_hook("PostToolUse", test_tracker_hook)
register_hook("Stop", quality_gate_hook)


# Agentloop
def agent_loop(messages:list):
    while True:
        response=client.messages.create(
            model=MODEL,system=SYSTEM,messages=messages,
            tools=TOOLS,max_tokens=8000
        )

        messages.append({"role":"assistant","content":response.content})
        print(f"模型回复的所有内容:\n{messages}\n")

        tool_calls=[
            block for block in response.content if block.type=="tool_use"
        ]
        print(f"这部分是所有工具块：\n{tool_calls}\n")

        if not tool_calls:                          # 模型不再调工具，想收工
            force = trigger_hooks("Stop", messages) # ← 新增：下班前，挂钩过一遍
            if force:
                messages.append({"role": "user", "content": force})
                continue                            #   挂钩有话说 → 注入，继续转
```

```
            return                                  #    挂钩没意见 → 真下班

        results=[]
        for block in tool_calls:
            blocked = trigger_hooks("PreToolUse", block)   # ← 改动：替代 check_permission
            if blocked:                     # ← 新增的两行
                results.append({"type": "tool_result",
                        "tool_use_id": block.id,
                        "content": "Permission denied."})
                continue

            handler=TOOL_HANDLERS.get(block.name)
            output=handler(**block.input) if handler else f"Unknown:{block.name}"
            trigger_hooks("PostToolUse", block, output)    # ← 新增：执行完，挂钩过一遍
            results.append({
                "type":"tool_result",
                "tool_use_id":block.id,
                "content":output
            })
        messages.append({"role":"user","content":results})


if __name__=="__main__":
    print("请输入一个问题，点击enter发送，点击q会退出。\n")
    history=[]
    while True:
        try:
            query=input("claude code>>")
            trigger_hooks("UserPromptSubmit", query)      # ← 新增：进 LLM 之前
        except (EOFError,KeyboardInterrupt):
            break
        if query.strip().lower() in("q","exit",''):
            break

        history.append({"role":"user","content":query})
        agent_loop(history)

        for block in history[-1]["content"]:
            if getattr(block,'type',None)=="text":
                print(block.text)
        print()
```